WEEK 3 · LESSON 17 OF 24 · HANDOUT

# Evaluating LLM Applications

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=6u996P09EZE — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyData Virginia 2025: Mastering LLMs — from Prompt Engineering to Agentic AI — https://www.youtube.com/watch?v=jmwLzX_ltbQ

🎓 **Go deeper:** EuroPython 2024: Is RAG all you need? A look at the limits of retrieval augmented generation — https://www.youtube.com/watch?v=9wk7mGB_Gp4

**Your goal in this lesson:** Build an evaluation set for an LLM app, score answers with automatic checks and an LLM judge, test refusal and citation behaviour, and use the results to decide what to change.

## "It looked good in the demo" is not evaluation

LLM apps fail in subtle ways: a wrong number, a confident invention, a refusal when the answer was available. Changing the prompt, the model or the retrieval can fix one case and break three others. You need a **repeatable test** you can run after every change — like the test sets in Machine Learning Fundamentals.

```
import importlib.util, os, subprocess, sys

if importlib.util.find_spec("anthropic") is None:          # install once per Colab session
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "anthropic"], check=True)

try:
    from google.colab import userdata                     # Colab: key saved in the Secrets
panel (key icon)
    os.environ["ANTHROPIC_API_KEY"] = userdata.get("ANTHROPIC_API_KEY")
except ImportError:
    pass                                                    # not in Colab: set
ANTHROPIC_API_KEY in your environment
except Exception:
    print("Add a secret called ANTHROPIC_API_KEY in Colab's Secrets panel, then run this cell
again.")

import anthropic

client = anthropic.Anthropic()        # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-haiku-5-5"            # fast and low-cost; check docs.claude.com for current
model names

def ask(prompt, system=None, max_tokens=500):
    extra = {"system": system} if system else {}
    response = client.messages.create(model=MODEL, max_tokens=max_tokens,
                                      messages=[{"role": "user", "content": prompt}], **extra)
    return response.content[0].text
```

```
HANDBOOK = {
    "Opening hours": "The Kigali Tech Hub is open Monday to Friday from 8:00 to 20:00 and on
Saturday "
                     "from 9:00 to 14:00. The hub is closed on Sundays and on public
holidays.",
    "Membership plans": "There are three plans. The Student plan costs 10,000 RWF per month and
needs a "
                        "valid student card. The Standard plan costs 25,000 RWF per month. The
Team plan "
                        "costs 90,000 RWF per month for up to five people.",
    "Joining": "To become a member, fill in the online form, bring your national ID or passport
to the "
               "front desk, and pay the first month. Your access card is ready the same day.",
    "Payment": "Fees are paid by mobile money or bank transfer before the 5th of each month.
Members who "
               "pay late for two months lose access until the balance is cleared.",
    "Cancelling": "You may cancel your membership at any time with 14 days' written notice by
email. "
                  "Fees already paid for the current month are not refunded.",
    "Wi-Fi": "The Wi-Fi network is called KTH-Members. The password is printed on your access
card and "
             "changes every three months. Guests use the separate KTH-Guest network.",
    "Meeting rooms": "Members can book the two meeting rooms through the hub app. Standard and
Team members "
                     "get 4 free hours per month; Student members get 1 free hour. Extra hours
cost 5,000 RWF.",
    "Guests": "Each member may bring up to two guests per day. Guests must sign in at the front
desk "
              "and leave by 18:00.",
    "Printing": "Black-and-white printing costs 50 RWF per page and colour printing 200 RWF per
page. "
                "Students get 100 free black-and-white pages per month.",
    "Events": "The hub runs a free coding workshop every Wednesday at 17:00 and a startup pitch
night "
              "on the last Friday of each month. Members can register in the hub app.",
    "Lockers": "Lockers can be rented for 3,000 RWF per month. Lost locker keys cost 5,000 RWF
to replace.",
    "Code of conduct": "Members must respect others, keep noise low in the quiet zone, and
never share their "
                       "access card. Harassment leads to immediate cancellation of
membership.",
    "Lost and found": "Items left behind are kept at the front desk for 30 days, then
donated.",
    "Accessibility": "The ground floor, meeting rooms and toilets are wheelchair accessible. A
lift serves "
                     "all floors. Contact the front desk to arrange other support.",
}
chunks = [{"id": i, "title": t, "text": f"{t}. {body}"} for i, (t, body) in
enumerate(HANDBOOK.items())]
print(len(chunks), "sections")
```

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import Normalizer

texts = [c["text"] for c in chunks]
embedder = make_pipeline(TfidfVectorizer(stop_words="english", sublinear_tf=True),
                         TruncatedSVD(n_components=12, random_state=0),
                         Normalizer())                      # unit length: dot product = cosine
similarity
doc_vectors = embedder.fit_transform(texts)

def embed(text_list):
    return embedder.transform(text_list)

print("each section is now a vector of", doc_vectors.shape[1], "numbers")
```

## The app under test

```
import re

RAG_SYSTEM = ("Answer using ONLY the numbered sources. Cite like [1]. If the sources do not
contain "
              "the answer, reply exactly: \"I don't know based on the handbook.\"")

def retrieve(question, k=3):
    return [chunks[i] for i in np.argsort(doc_vectors @ embed([question])[0])[::-1][:k]]

def rag_answer(question):
    sources = retrieve(question)
    numbered = "\n".join(f"[{n}] {s['text']}" for n, s in enumerate(sources, 1))
    text = ask(f"<sources>\n{numbered}\n</sources>\n\n<question>{question}</question>",
               system=RAG_SYSTEM, max_tokens=300)
    return text, sources
```

## An evaluation set

Each case has a question, **facts the answer must contain**, and whether the app should refuse:

```
eval_set = [
    {"q": "How much is the Student plan per month?", "must_include": ["10,000"], "refuse":
False},
    {"q": "Until what time is the hub open on Saturday?", "must_include": ["14:00"], "refuse":
False},
    {"q": "How many free meeting-room hours do Student members get?", "must_include": ["1"],
"refuse": False},
    {"q": "How long is lost property kept?", "must_include": ["30"], "refuse": False},
    {"q": "What is the price of a coffee at the hub cafe?", "must_include": [], "refuse":
True},
    {"q": "Who is the President of the hub's board?", "must_include": [], "refuse": True},
]
print(len(eval_set), "test cases")
```

Include **unanswerable** questions on purpose: they test whether the app admits it does not know.

## Automatic checks

```
REFUSAL = "i don't know based on the handbook"

def check(case, answer, sources):
    text = answer.lower()
    refused = REFUSAL in text
    result = {"refusal_ok": refused == case["refuse"]}
    if not case["refuse"]:
        result["facts_ok"] = all(fact.lower() in text for fact in case["must_include"])
        cited = [int(n) for n in re.findall(r"\[(\d+)\]", answer)]
        result["cites_ok"] = bool(cited) and all(1 <= n <= len(sources) for n in cited)
    return result

rows = []
for case in eval_set:
    answer, sources = rag_answer(case["q"])
    rows.append({"q": case["q"][:45], **check(case, answer, sources)})

import pandas as pd
report = pd.DataFrame(rows)
print(report)
print({c: f"{report[c].dropna().mean():.0%}" for c in report.columns if c != "q"})
```

Three simple, objective checks: Did it refuse when it should (and only then)? Does the answer contain the required facts? Does every citation point to a real source?

## LLM-as-judge for the rest

Some qualities — is the answer complete, polite, faithful to the sources? — are hard to check with code. You can ask a **second model call** to grade with a clear rubric:

```
JUDGE = ("You grade answers. Given SOURCES, QUESTION and ANSWER, score from 1 to 5: "
         "5 = fully correct and supported by the sources, 3 = partly correct or missing
details, "
         "1 = wrong or not supported. Reply in the form:\nSCORE: <number>\nREASON: <one
sentence>")

def judge(question, answer, sources):
    numbered = "\n".join(f"[{n}] {s['text']}" for n, s in enumerate(sources, 1))
    verdict = ask(f"SOURCES:\n{numbered}\n\nQUESTION: {question}\n\nANSWER: {answer}",
system=JUDGE, max_tokens=100)
    match = re.search(r"SCORE:\s*([1-5])", verdict)
    return (int(match.group(1)) if match else None), verdict

answer, sources = rag_answer(eval_set[0]["q"])
score, verdict = judge(eval_set[0]["q"], answer, sources)
print("judge score:", score)
```

Judges are useful but imperfect: they can be too kind, prefer longer answers, or share the same blind spots as the model being judged. **Check a sample of judge scores by hand**, and prefer objective checks where possible.

## Use the results

- Look at **every failure**, not just the average. Group failures: retrieval missed? Wrong number? Refused wrongly?
- Change **one thing at a time** (prompt, k, chunking, model), re-run the whole set, and compare.
- Keep the evaluation set growing: every bug reported by a real user becomes a new test case.

- For languages such as Kinyarwanda, build test cases in that language — quality can be very different from English.

## Key terms

- **Evaluation set** — questions with expected facts or behaviour, run after every change.
- **Refusal test** — an unanswerable question that should produce "I don't know".
- **Citation check** — verifying citations point to real retrieved sources.
- **LLM-as-judge** — using a model with a rubric to grade answers.
- **Regression** — something that used to work and broke after a change.

**Remember this:** Evaluate LLM apps like software: a growing test set with objective checks, refusal tests and a carefully checked LLM judge, re-run after every change.

Name: _____ Date: _____

## Activity

### Task 1

Add five more cases, including two in Kinyarwanda. Which checks fail?

### Task 2

Change `k` in `retrieve` from 3 to 1. Which scores drop?

### Task 3

Ask the judge to grade a deliberately wrong answer ("The Student plan costs 50,000 RWF [1]"). Does it notice?

### Task 4

Write the evaluation as a function you can run with one line after each change, and save its results with the date.

## Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why is a demo not enough to evaluate an LLM app?
   A. LLMs cannot be tested
   B. Changes can fix one case and break others; you need a repeatable test set
   C. Demos are illegal
   D. Demos cost too much
2. Why include unanswerable questions in the evaluation set?
   A. To increase cost
   B. They are not useful
   C. To confuse the model
   D. To test that the app admits when it does not know
3. What does a citation check verify?
   A. Response time
   B. That citations point to real retrieved sources
   C. Token counts
   D. Spelling
4. What is an LLM-as-judge?
   A. A model call that grades answers with a rubric
   B. An embedding
   C. A legal system
   D. A human expert
5. What is a weakness of LLM judges?
   A. They need GPUs
   B. They cannot read
   C. They are always perfect
   D. They can be too kind or share blind spots, so sample their scores by hand
6. What should you do with a bug reported by a real user?
   A. Delete the app
   B. Ignore it
   C. Change the model at random
   D. Add it to the evaluation set as a new test case

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.